Engineering Seat Allocation Software

ICS1261- Software Development and Practice using Python

A PROJECT REPORT

Submitted By

PANDIARAJAN D 3122237001035
RUSHABH S 3122237001044

5-Year Integrated M.Tech

Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

Kalavakkam – 603110

June 2024

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled "**Engineering Seat Allocation Software**" is the Bonafide work of "**PANDIARAJAN D (3122237001035)** and **RUSHABH S (3122237001044)**" who carried out the project work in the **ICS1261- Software Development and Practice using Python** during the academic year **2023-24**.

Internal Examiner External Examiner

Date :

Table of Contents:

INTRODUCTION-PROBLEM STATEMENT

Develop a software system for the engineering counselling and admission process for two sets of institutes (for example, say IITs and NITs) each having a set of different branches, each branch with a certain number of seats available. Number of candidates can be assumed as 5 times the total number of seats available. Each candidate can provide a list of preferences where each preference is a 2-tuple, (institute, branch). Admission to each set of institutes is based on its own qualifying exam (for example, JEE-Advanced and JEE-Main). Each candidate will have a specific rank in one or both merit lists.

Constraints:

- Seat allotment for a candidate must be from the list of choices given by the candidate.
- Number of preferences given by each candidate is limited to the number of institutes times the number of branches in each institute.
- Each candidate must be allotted only one of his/her choices.
- All the available seats in all the branches in all the Institutes must be filled.
- If a student is denied a particular choice, then all those who were allotted that choice must be higher in the respective rank list (Merit should not be violated).

Input:

- Number of institutes
- Number of branches in each institute
- Number of seats for each branch in each institute
- Rank list of candidates for each qualifying exam

Output:

Generation of rank list based on merit and allocation of institute and branch to each candidate as per policy.

EXECUTIVE SUMMARY/EXTENDED EXPLORATION:

- The aim of the outlined software system is to **streamline and automate the engineering counselling and admission process**.
- It involves developing various components, including a comprehensive database to store **candidate information**, **institute details**, **available seats**, **rank**, and **marks**.
- Additionally, the system will incorporate modules for generating **random candidate information**, **generating rank lists**, and **generating results**.
- By integrating these functionalities, the software aims to enhance efficiency, transparency, and accuracy in the admission process, ultimately facilitating smoother operations for both candidates and institutions involved.
- This project helps us to enhance our technical abilities in python, algorithm design, and software development.
- This project helps us to improve our problem-solving skills by addressing challenges in preference processing (caste based), seat allocation, and result generation.

SEQUENCE OF STEPS IN GENERATING RANK LIST:

• Generating random information (DOB, caste, marks) of the candidates.

• Calculating cutoff marks using the marks generated.

• Using the below **criteria**(weightage) the new value is decided:

Caste

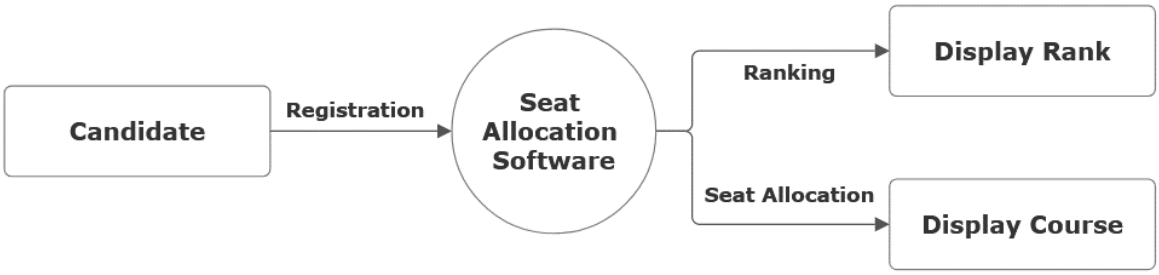
OC => 0.92
BC => 0.94
MBC => 0.96
SC => 0.98
ST => 1.00

• With the new value calculated, rank list is generated.

ANALYSIS USING DATA FLOW AND CONTROL FLOW DIAGRAM:

DATA FLOW DIAGRAM :

Level 0 Data Flow Diagram (Context Diagram) :

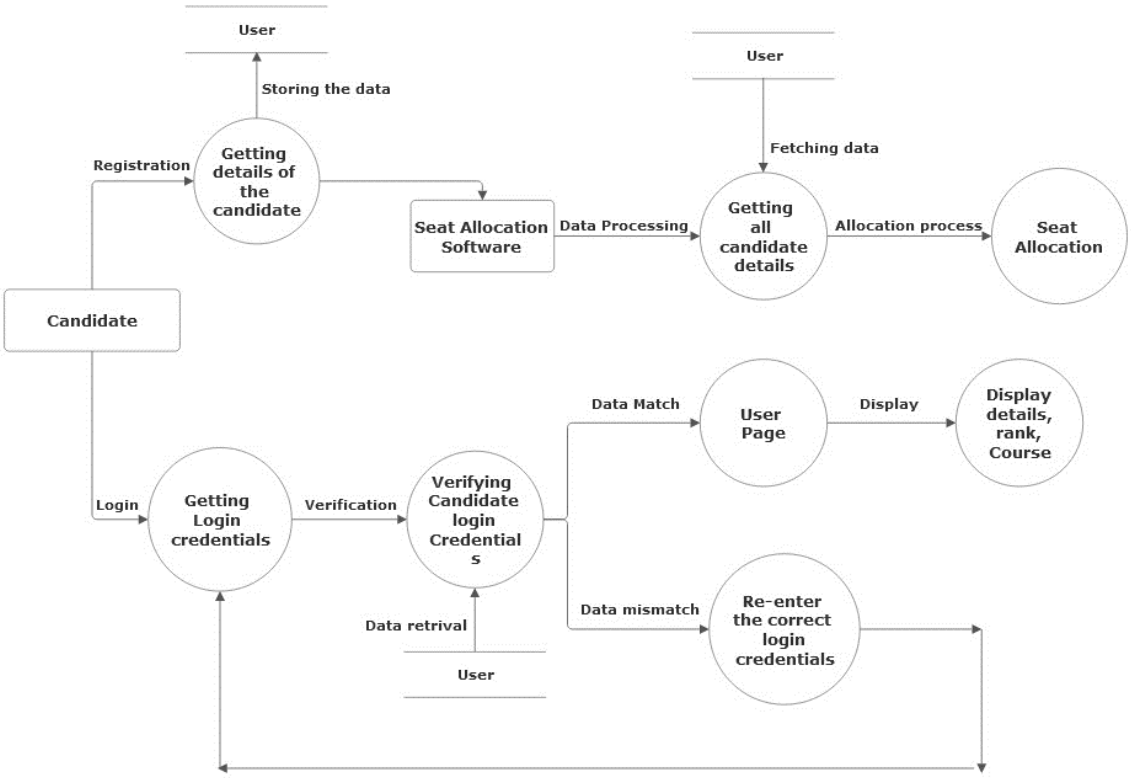


Here the Seat allocation Software system is the main process into which the Candidates information goes into and then it ranks all the Candidates and allocates the seats depending on their Choice filling.

The Overall main modules are

- Registration / Login Module
- Ranking Module
- Seat Allocation Module

Level 1 Data Flow Diagram:



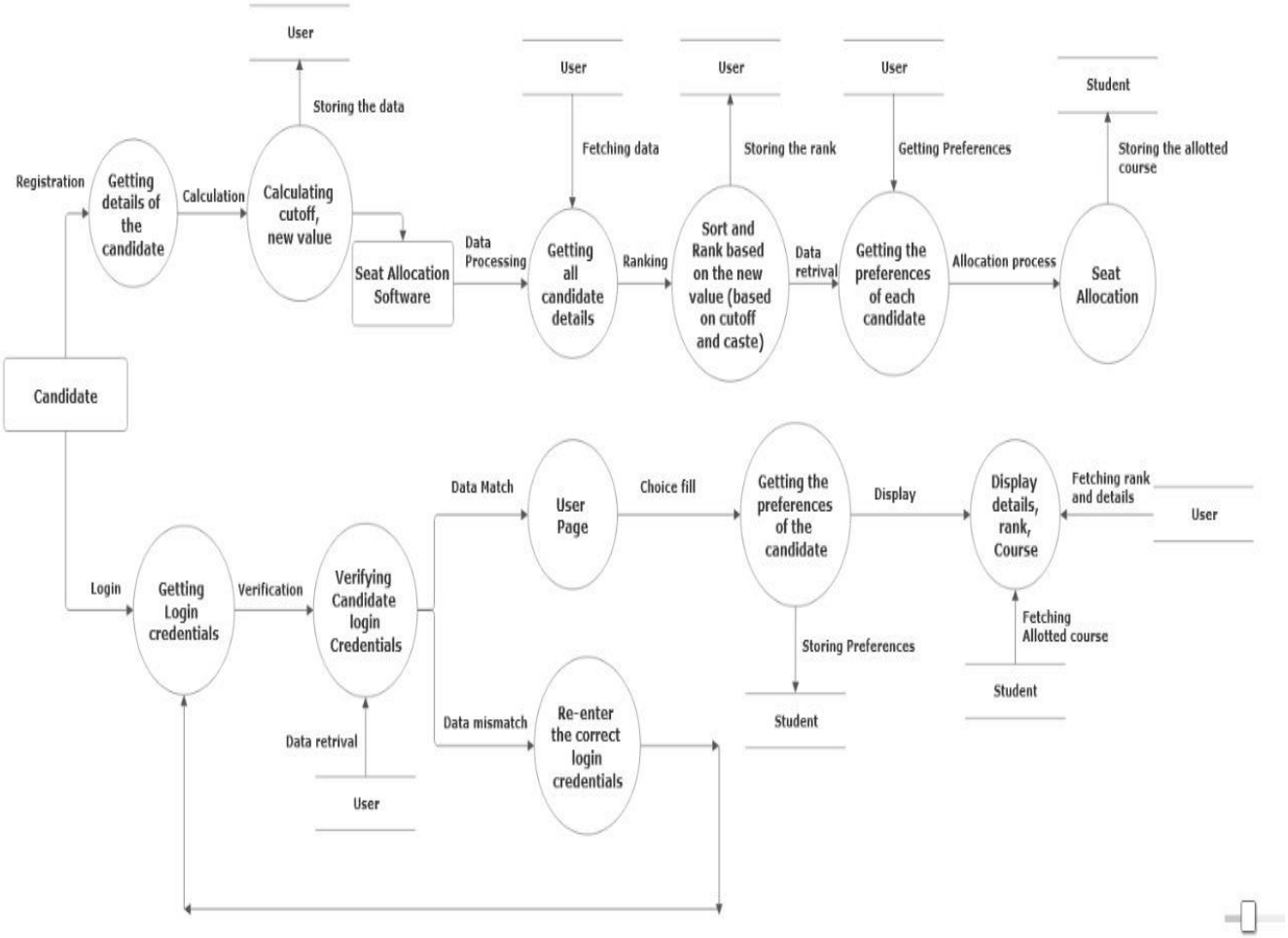
- Here the Registration Module gets the input from the candidates and stores them in the ‘user’ database.

- If the Registered candidate wants to login, login Module gets the login credentials from the Candidate and checks with the database and allows him/her to login if it matches or else again prompts to enter login credentials.

- Now the Seat Allocation Software fetch the data from the database table ‘user’ and allot the seat to all candidate based on their cutoff, caste and their choice filling.

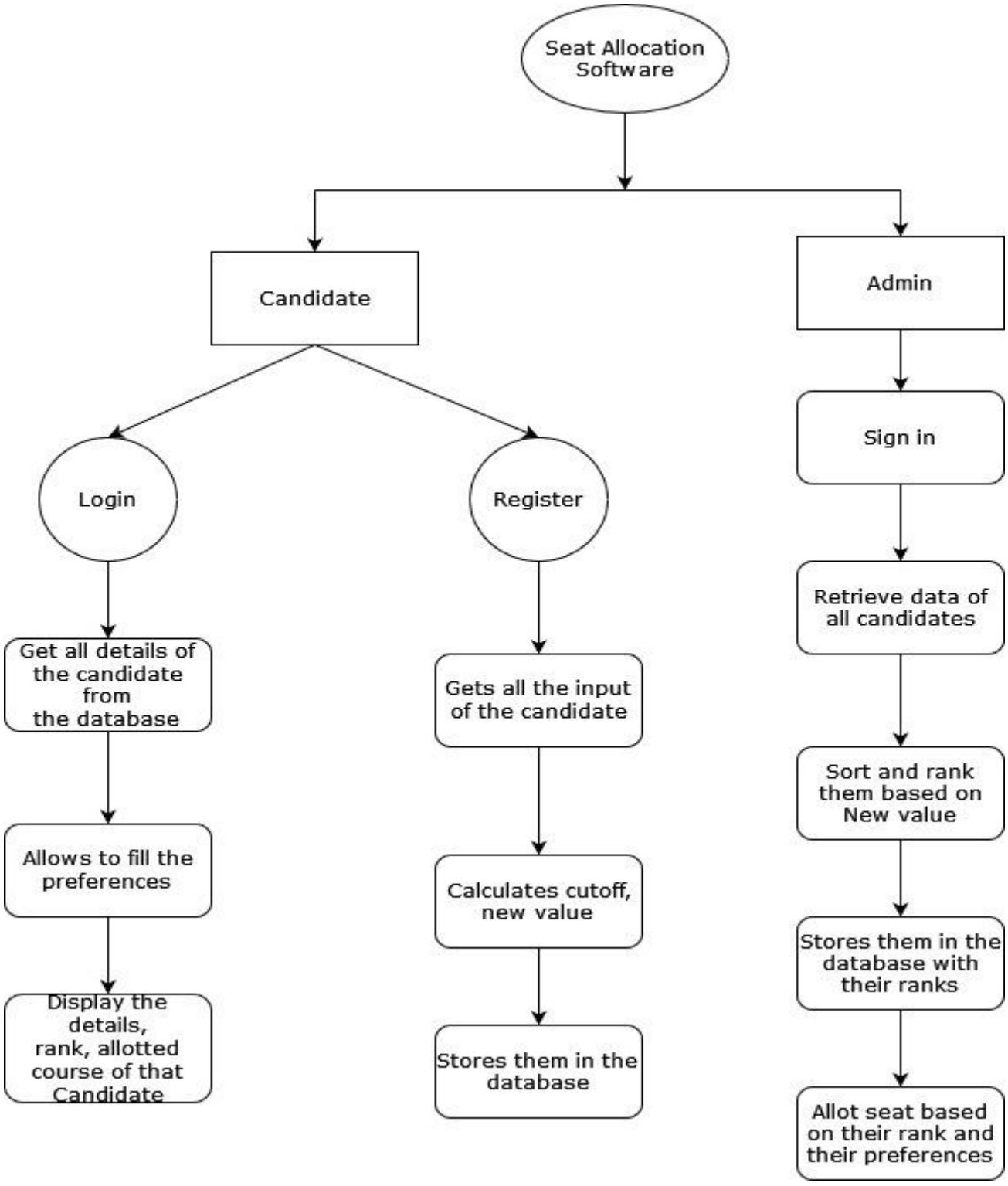
- Finally, the allocated seats are updated to the database and displayed while Candidate log in

Level 2 Data Flow Diagram:



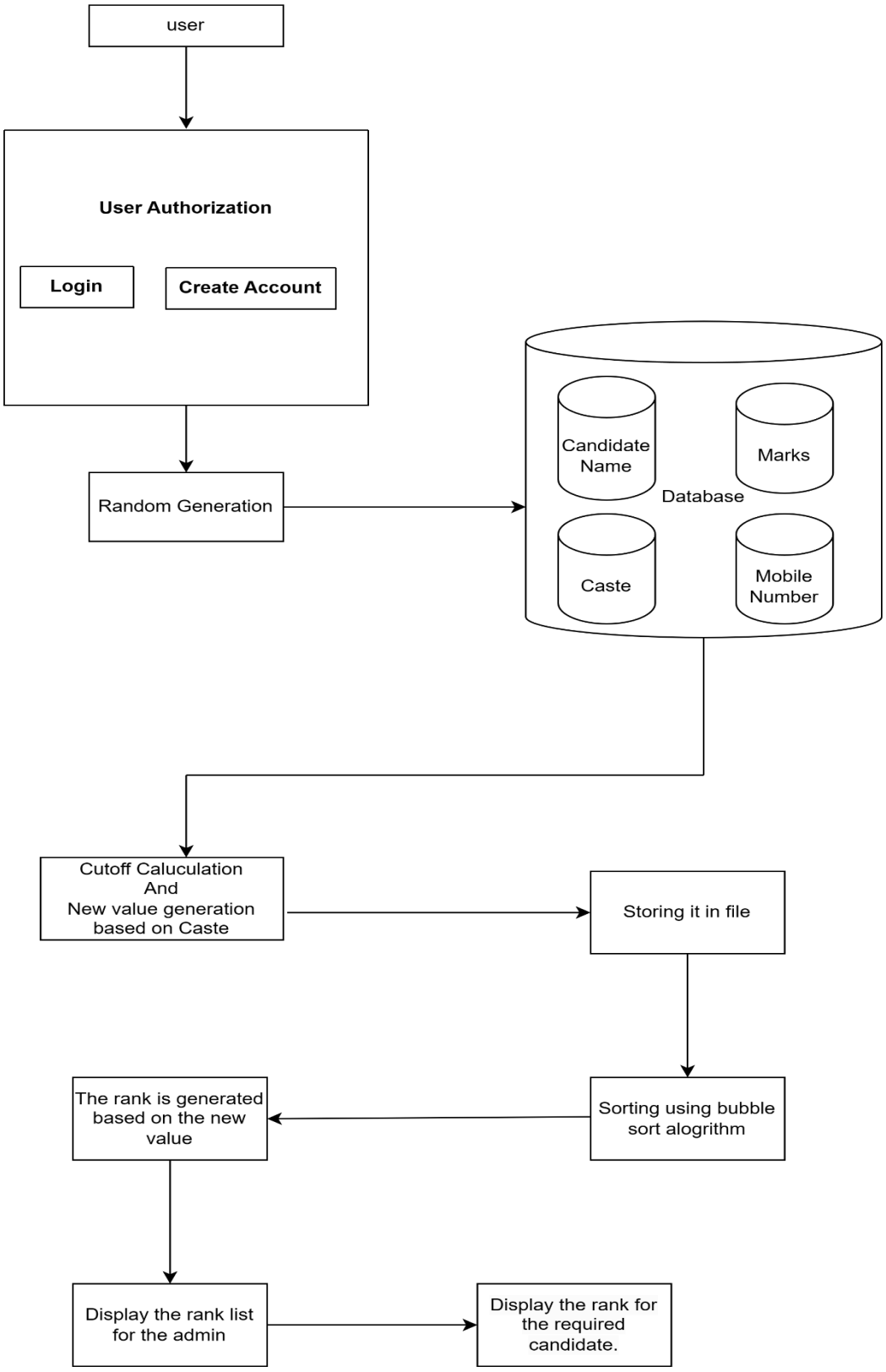
- The Level 2 data flow diagram provides the detailed overview and explains all process of the Seat Allocation Software.

CONTROL FLOW DIAGRAM :



UML Diagrams:

1. Architecture Diagram:



Visual representation of the system's components and their interactions.

Modules Identified:

This represents how the modules of the system have been integrated and a detailed breakdown of the code modules with the help of structure/flow diagram.

1. Login Module

2. New Registration Module

3. Random generation Module

4. Cutoff calculation

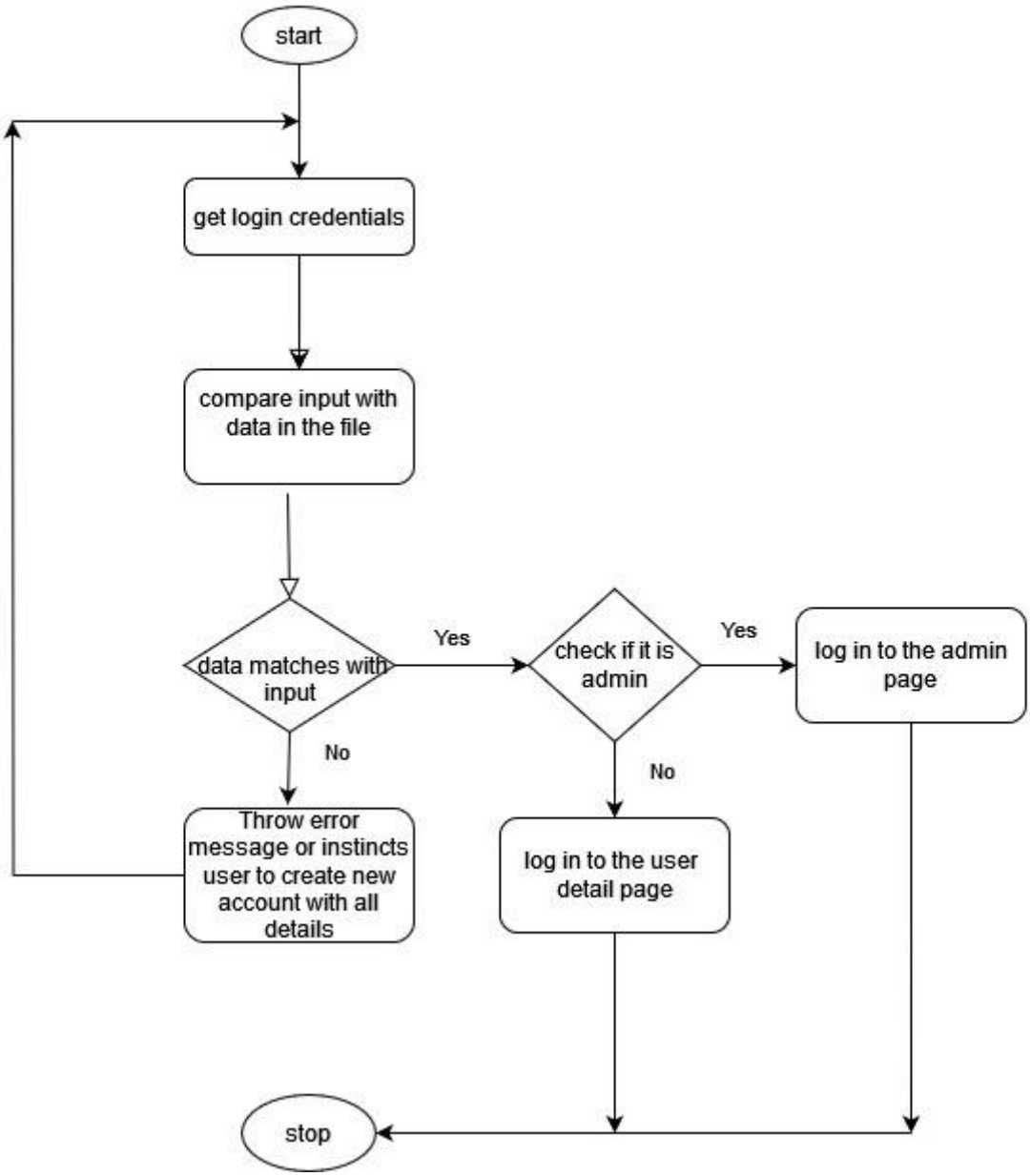
5. Rank generation

6. Seat Allotment

7. Display Result

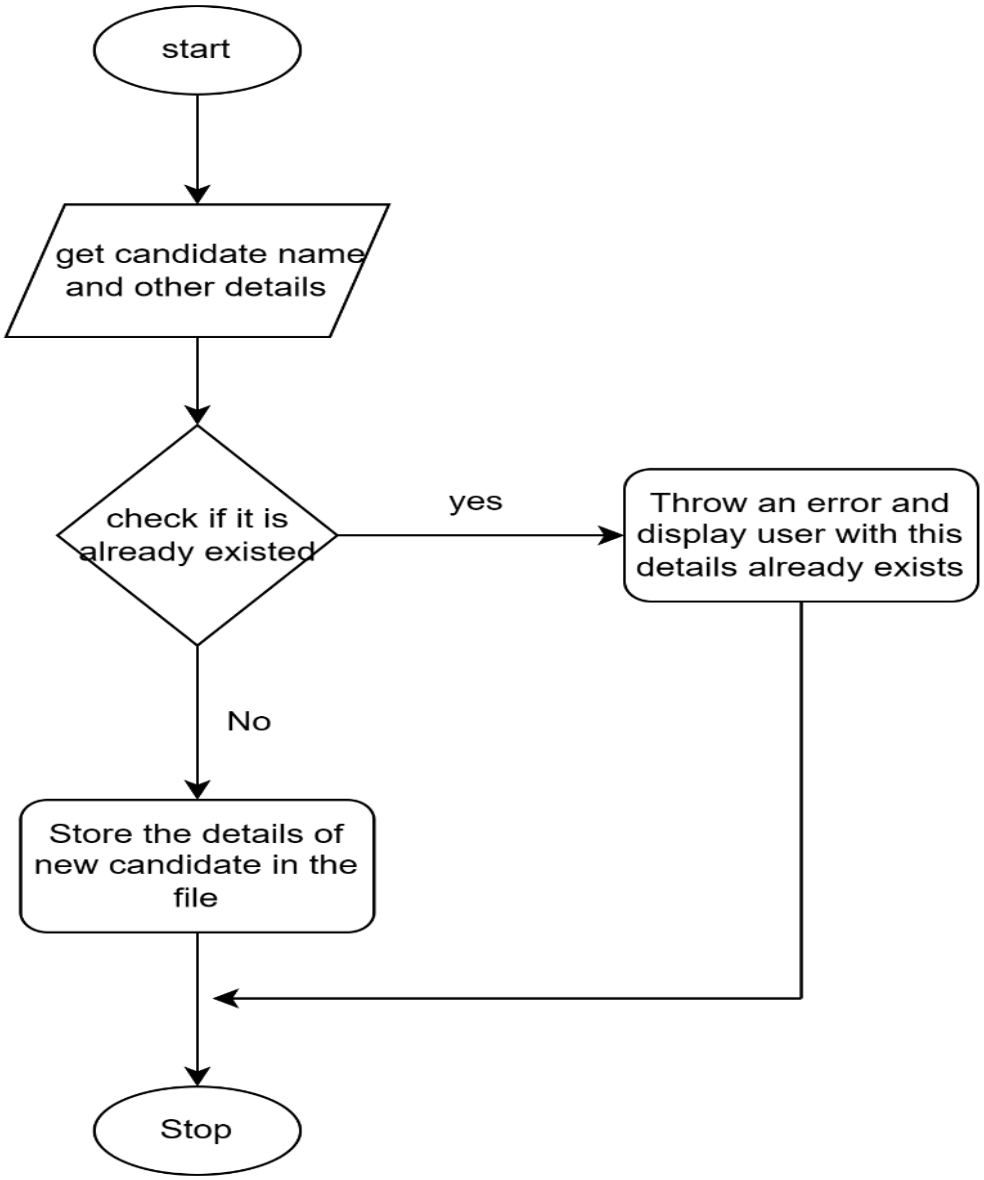
Login Module:

This module shows a user login and registration process. It starts by checking if the user is new and requires registration. For existing users, it verifies their credentials against stored data and determines if they have admin privileges, directing them to the appropriate admin or user page accordingly. If credentials don't match, it prompts for error handling or account creation.



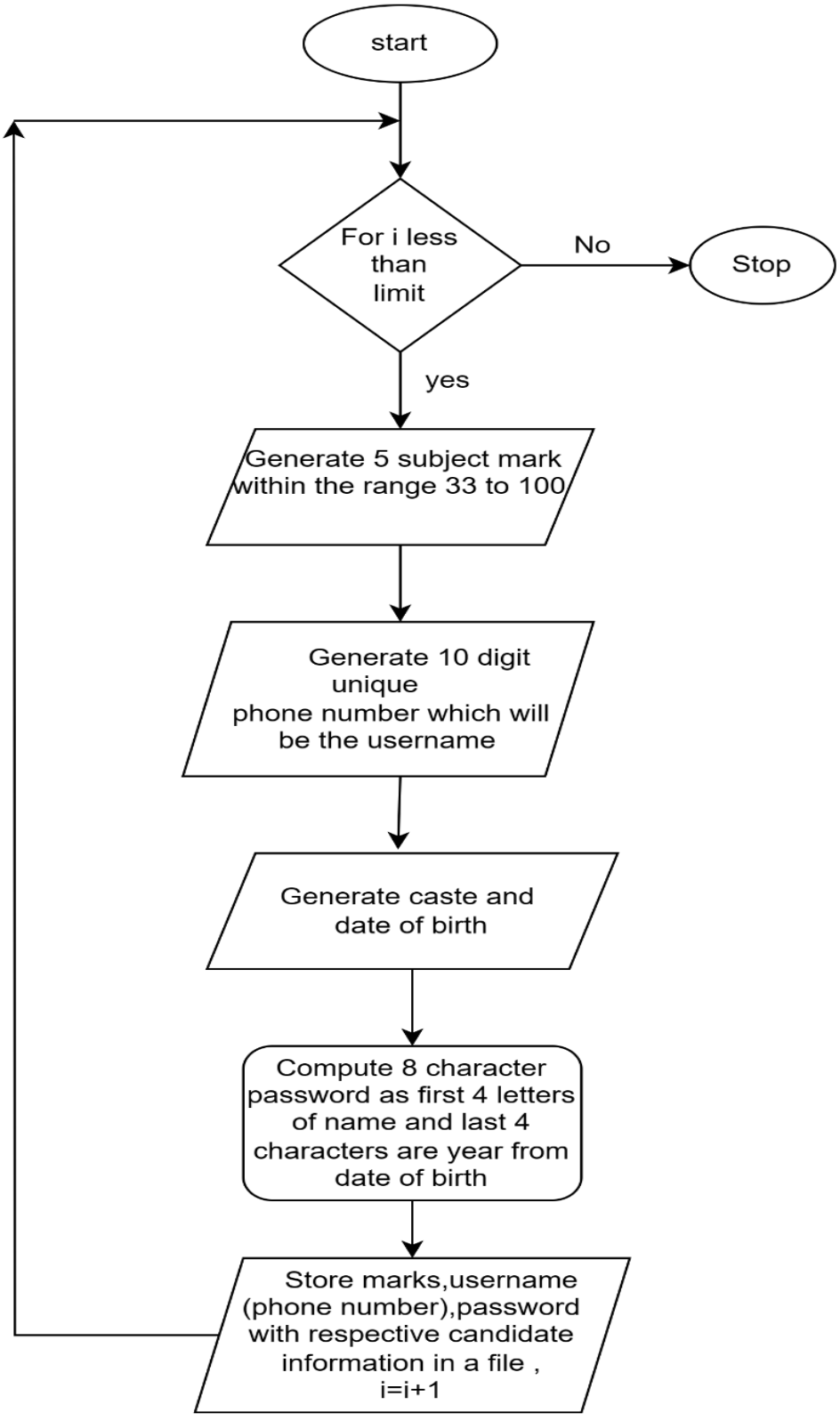
New Registration Module:

This module describes a candidate registration process. It starts by collecting the candidate's name and details, then checks if these details already exist in the system. If the candidate is already registered, an error is displayed. If not, the new candidate's details are stored in the file, and the process ends.



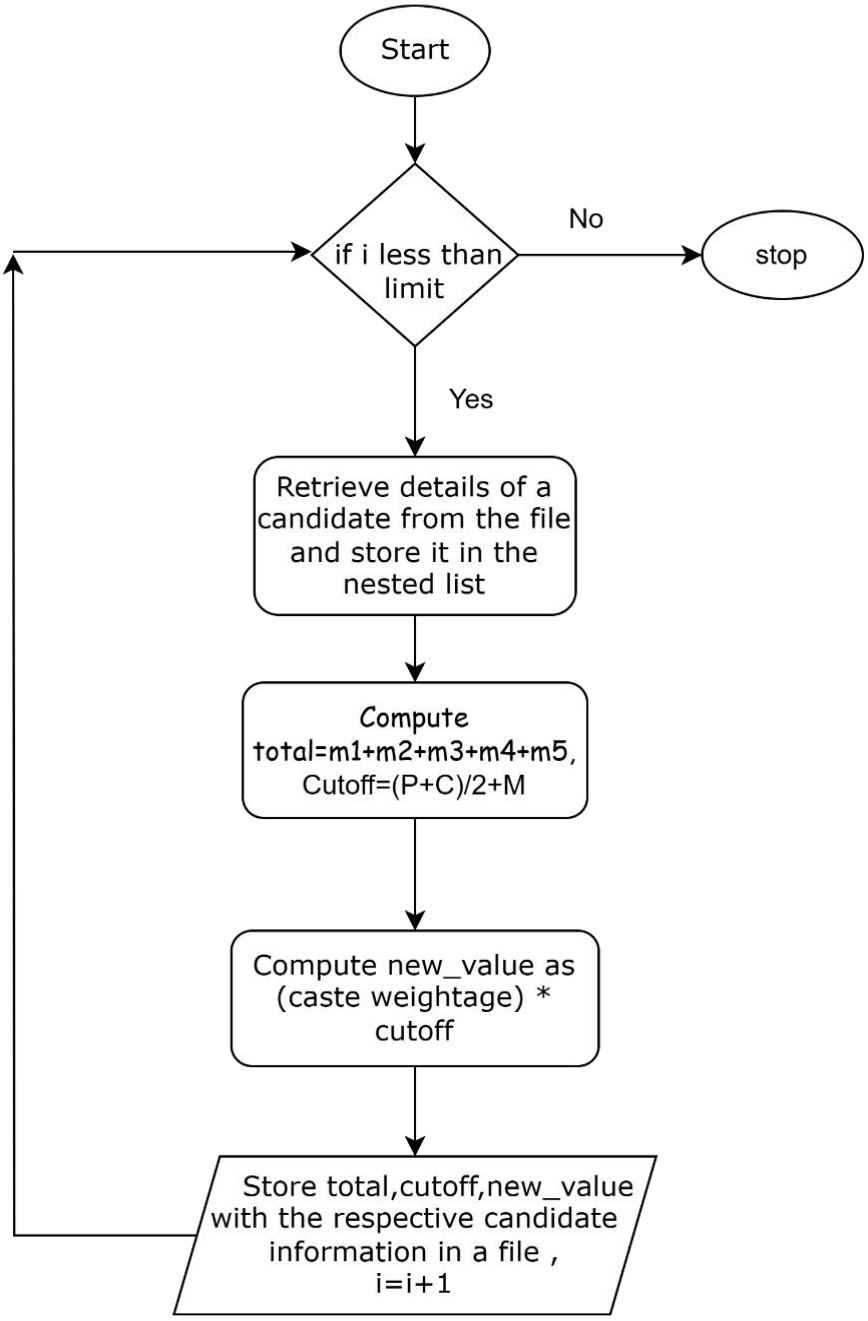
Random generation Module:

This flowchart automates generating and storing candidate information. It loops to create marks, a unique phone number (username), caste, date of birth, and an 8-character password, then stores these details for each candidate until a specified limit is reached. The process stops once the limit is met.



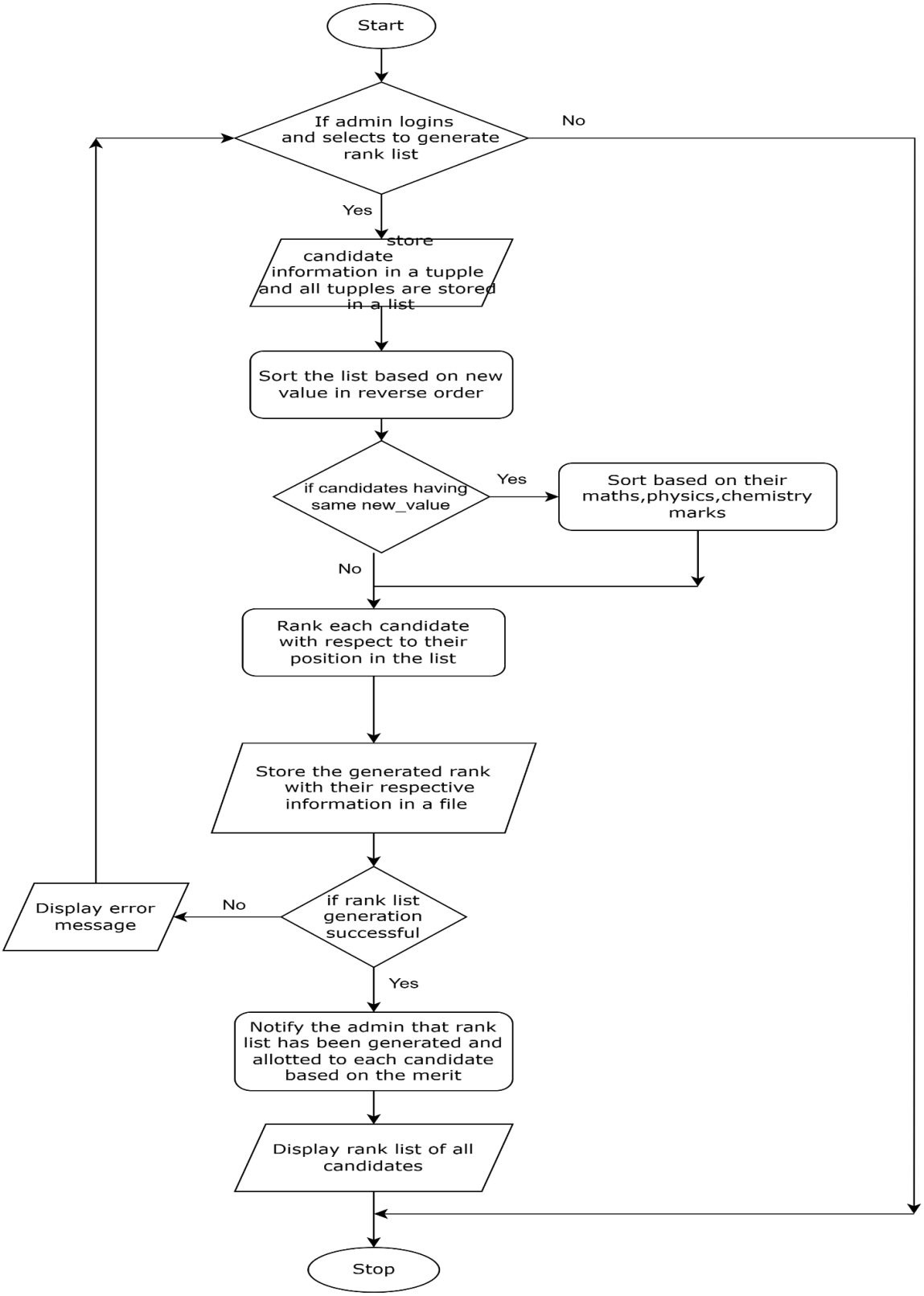
Cutoff calculation:

This module outlines a process for iterating through candidate details from a file up to a specified limit. For each candidate, it retrieves their details, computes total marks and a cutoff value, calculates a new value using a caste weightage multiplier, and then stores these computed values along with the candidate's information in a file. The process increments the counter and repeats until the limit is reached.



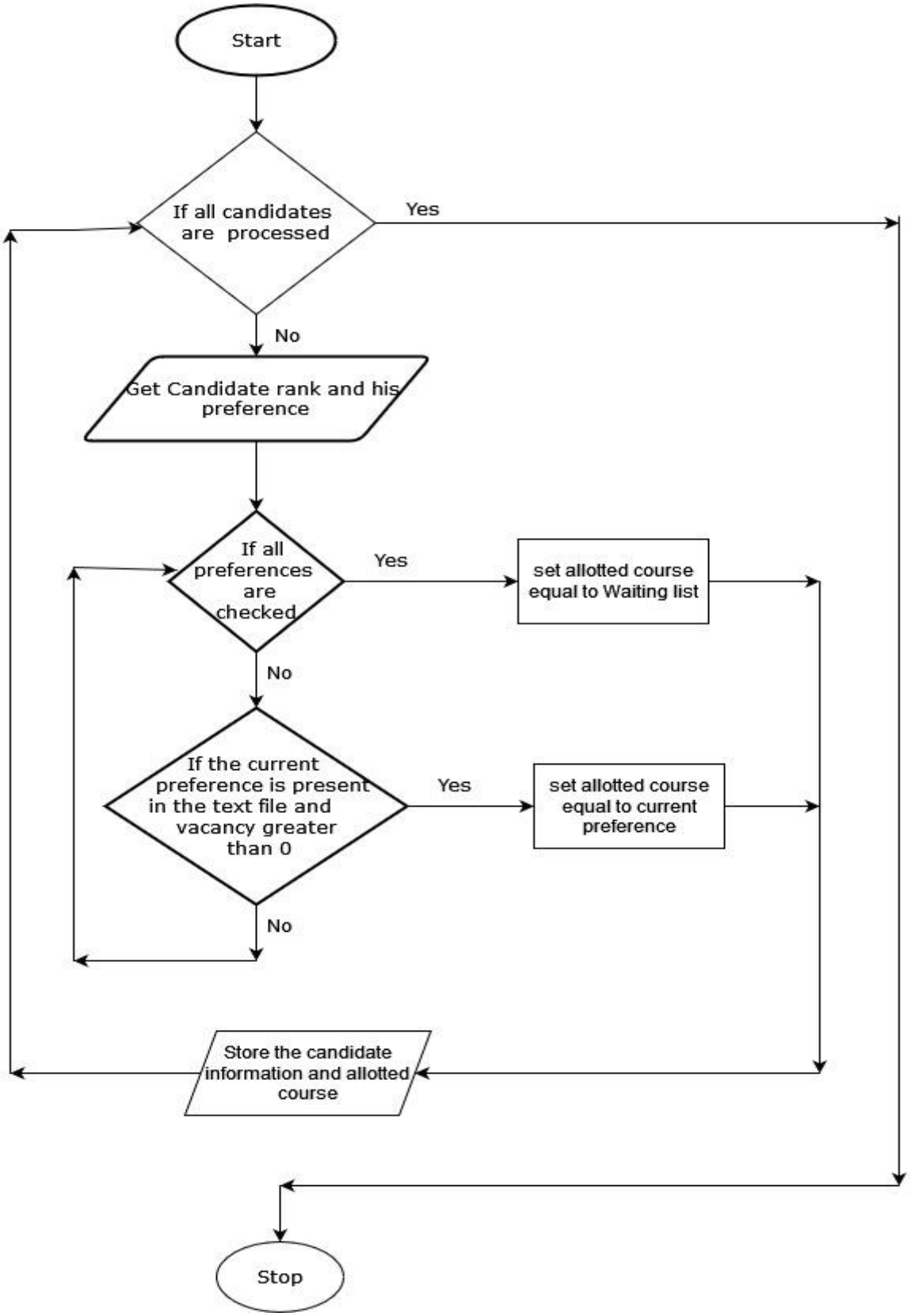
Rank generation:

This module describes the process of generating a rank list for candidates. Upon admin login, candidate details are stored in a list and sorted by new_value in descending order. If there are ties, candidates are further sorted by their math, physics, and chemistry marks. Each candidate is ranked, and the list is stored in a file. If successful, the admin is notified and the rank list is displayed; otherwise, an error message is shown.



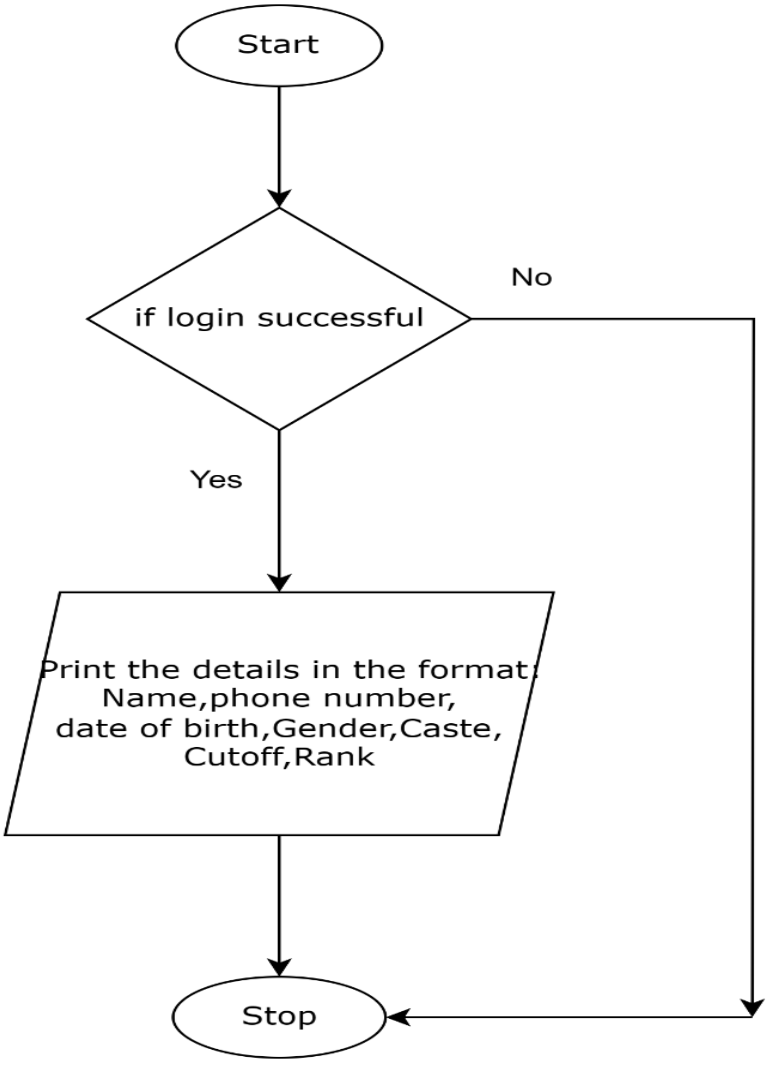
Seat Allotment :

This module describes the process of seat allocation for candidates and storing them in the database.



Display Result:

This module outlines the process for printing candidate details. Upon a successful login, the system prints candidate information in a specific format, including Name, Phone Number, Date of Birth, Gender, Caste, Cutoff, and Rank, then the process stops.



EXPLANATION OF DATA ORGANIZATION AND LANGUAGE CONSTRUCTS

- This project using many datatypes like list, tuple, classes, etc.
- The sorting which is based on the new value does the sorting and stores it in the form of a nested list which has the information of candidates.
- Classes are utilized while displaying the rank list of candidates.
- The lower number of selection of lists and tuples allows efficient storage and lower stress on memory.

RATIONALE:

- Lists provide a convenient way to store and access multiple instances of data, such as student details, choice filling preferences, etc.

```
cursor.execute("SELECT * FROM user ORDER BY unvalue DESC, um1 DESC, um2 DESC, um3
DESC, utotal DESC,uyear ASC")
            data=cursor.fetchall()
            data=list(data)
```

- Classes provide structure of columns for tables by specifying the datatype of the columns.

```
class User(models.Model):

    uname=models.CharField(max_length=30)
    upass=models.CharField(max_length=25,default='')
    uemail=models.EmailField()
    ucaste=models.CharField(max_length=3, choices=CASTE_CHOICES,default='OC')
    uyear=models.PositiveIntegerField(choices=DOB_CHOICES,default='2005')
    um1=models.PositiveSmallIntegerField(default=35)
    um2=models.PositiveSmallIntegerField(default=35)
    um3=models.PositiveSmallIntegerField(default=35)
    um4=models.PositiveSmallIntegerField(default=35)
    utotal=models.PositiveIntegerField(default=140)
    ucutoff=models.DecimalField(max_digits=5, decimal_places=2, default=0.00)
    unvalue=models.DecimalField(max_digits=5, decimal_places=2, default=0.00)
    urank=models.PositiveIntegerField(default=0)
    uper=models.DecimalField(max_digits=5, decimal_places=2, default=0.00)
```

```
    def __str__(self):
        return self.uname

    class Meta:
        db_table = "user"

class Student(models.Model):
    name = models.CharField(max_length=100)
    marks = models.DecimalField(max_digits=5, decimal_places=2)
    rank = models.PositiveSmallIntegerField(default=0)
    pref1 = models.CharField(max_length=250)
    pref2 = models.CharField(max_length=250)
    pref3 = models.CharField(max_length=250)
    allotted_course = models.CharField(max_length=250,default='Not allotted')
    #branch = models.CharField(max_length=100, blank=True, null=True)

    def __str__(self):
        return self.name

    class Meta:
        db_table = "student"
```

- Files have been used for storing the candidate's name to populate the database and to have a count of number of seats in a particular college and course.File handling operations enable the program to read and write data to external files, ensuring data persistence and allowing for the management of student name and allotted course records.

```
def write(users):
    with open('myapp/details.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['Username', 'Email', 'CONTACT NUMBER', 'Caste', 'Year',
'Mark 1', 'Mark 2', 'Mark 3', 'Mark 4', 'Total', 'Cutoff', 'N-Value', 'Rank'])
        for user in users:
            writer.writerow([user.uname, user.uemail, user.id, user.ucaste,
user.uyear, user.um1, user.um2, user.um3, user.um4, user.utotal, user.ucutoff,
user.unvalue, user.urank])
```

- Tuples are used when we retrieve the information from database.

By employing these language constructs, the code achieves better organization, modularity, and scalability, enhancing the overall functionality of the cab booking system.

Platforms used for code development:

- Notepad
- Visual Studio code
- MySQL
- MS Excel

Students Info Database ‘user’ (Stored in MySQL):

```
mysql> use mydjangodb;
Database changed
mysql> desc user;
+---------+-------------------+------+-----+---------+----------------+
| Field   | Type              | Null | Key | Default | Extra          |
+---------+-------------------+------+-----+---------+----------------+
| id      | bigint            | NO   | PRI | NULL    | auto_increment |
| uname   | varchar(30)       | NO   |     | NULL    |                |
| uemail  | varchar(254)      | NO   |     | NULL    |                |
| ucaste  | varchar(3)        | NO   |     | NULL    |                |
| um1     | smallint unsigned | NO   |     | NULL    |                |
| um2     | smallint unsigned | NO   |     | NULL    |                |
| um3     | smallint unsigned | NO   |     | NULL    |                |
| um4     | smallint unsigned | NO   |     | NULL    |                |
| utotal  | int unsigned      | NO   |     | NULL    |                |
| uyear   | int unsigned      | NO   |     | NULL    |                |
| unvalue | decimal(5,2)      | NO   |     | NULL    |                |
| upass   | varchar(25)       | NO   |     | NULL    |                |
| urank   | int unsigned      | NO   |     | NULL    |                |
| ucutoff | decimal(5,2)      | NO   |     | NULL    |                |
+---------+-------------------+------+-----+---------+----------------+
14 rows in set (0.21 sec)
```

Student's Preference Database 'student' (stored in mysql)

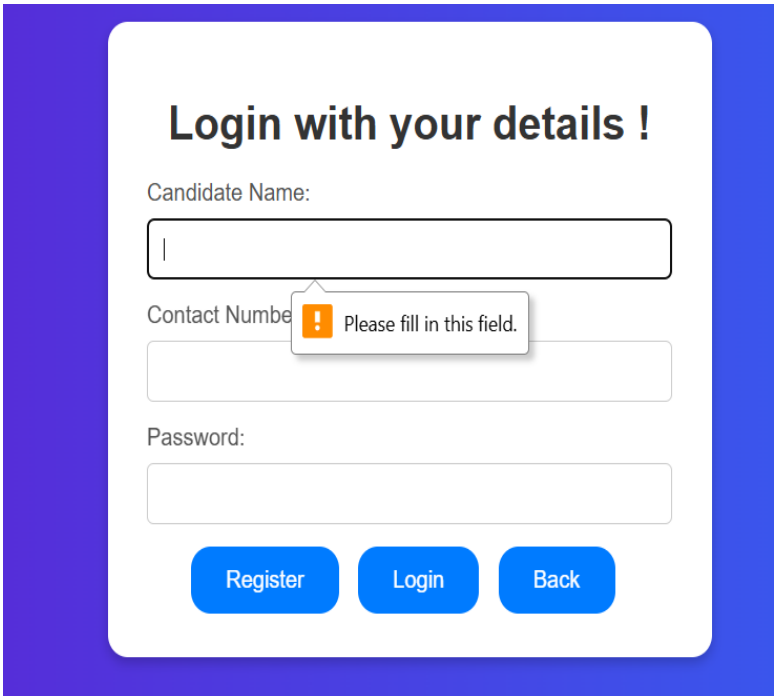
```
mysql> desc student;
+-----------------+-------------------+------+-----+---------+----------------+
| Field           | Type              | Null | Key | Default | Extra          |
+-----------------+-------------------+------+-----+---------+----------------+
| id              | bigint            | NO   | PRI | NULL    | auto_increment |
| name            | varchar(100)      | NO   |     | NULL    |                |
| marks           | decimal(5,2)      | NO   |     | NULL    |                |
| pref1           | varchar(250)      | NO   |     | NULL    |                |
| pref2           | varchar(250)      | NO   |     | NULL    |                |
| pref3           | varchar(250)      | NO   |     | NULL    |                |
| rank            | smallint unsigned | NO   |     | NULL    |                |
| allotted_course | varchar(250)      | NO   |     | NULL    |                |
+-----------------+-------------------+------+-----+---------+----------------+
8 rows in set (0.03 sec)
```

Validation through Detailed Test cases:

Login Module Test Cases:

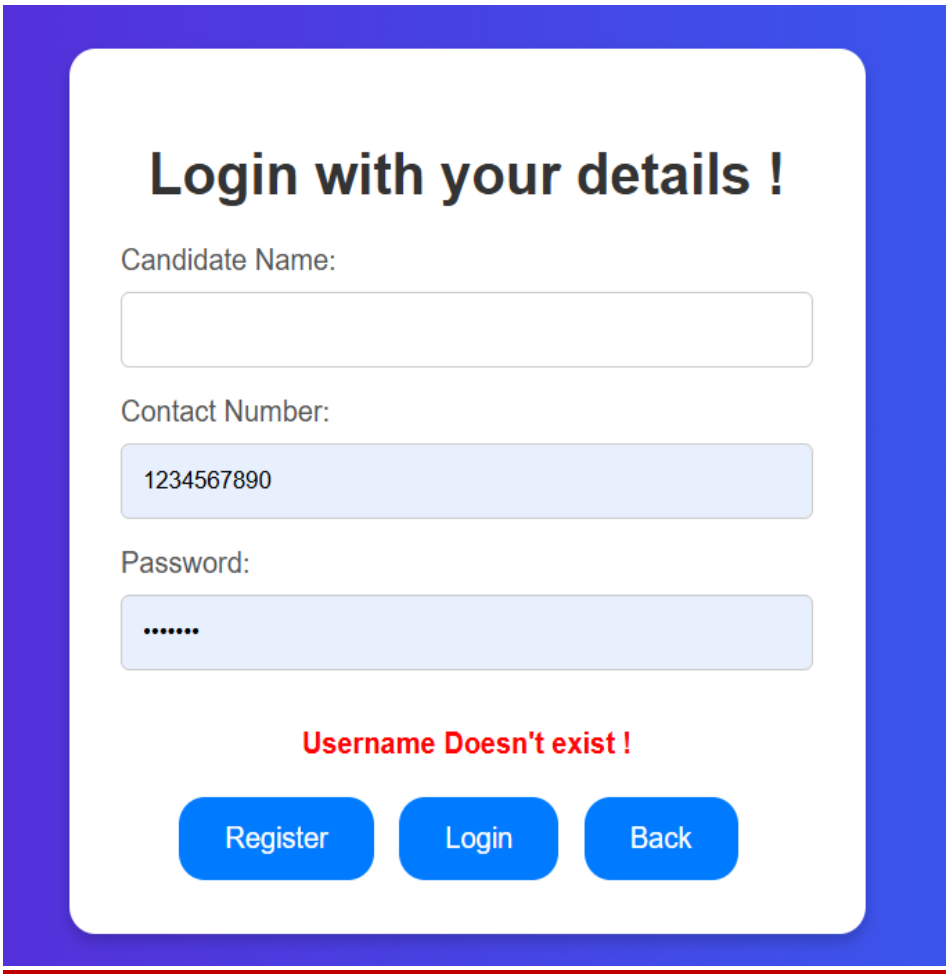
Test Case : Empty Inputs

- **Input:** Candidate Name : " ", Contact number : " ", Password : " ",
- **Expected Output**: Prompt user to fill in the form with a message "Please fill in this field"
- **Actual Output**:



Test Case : Wrong inputs

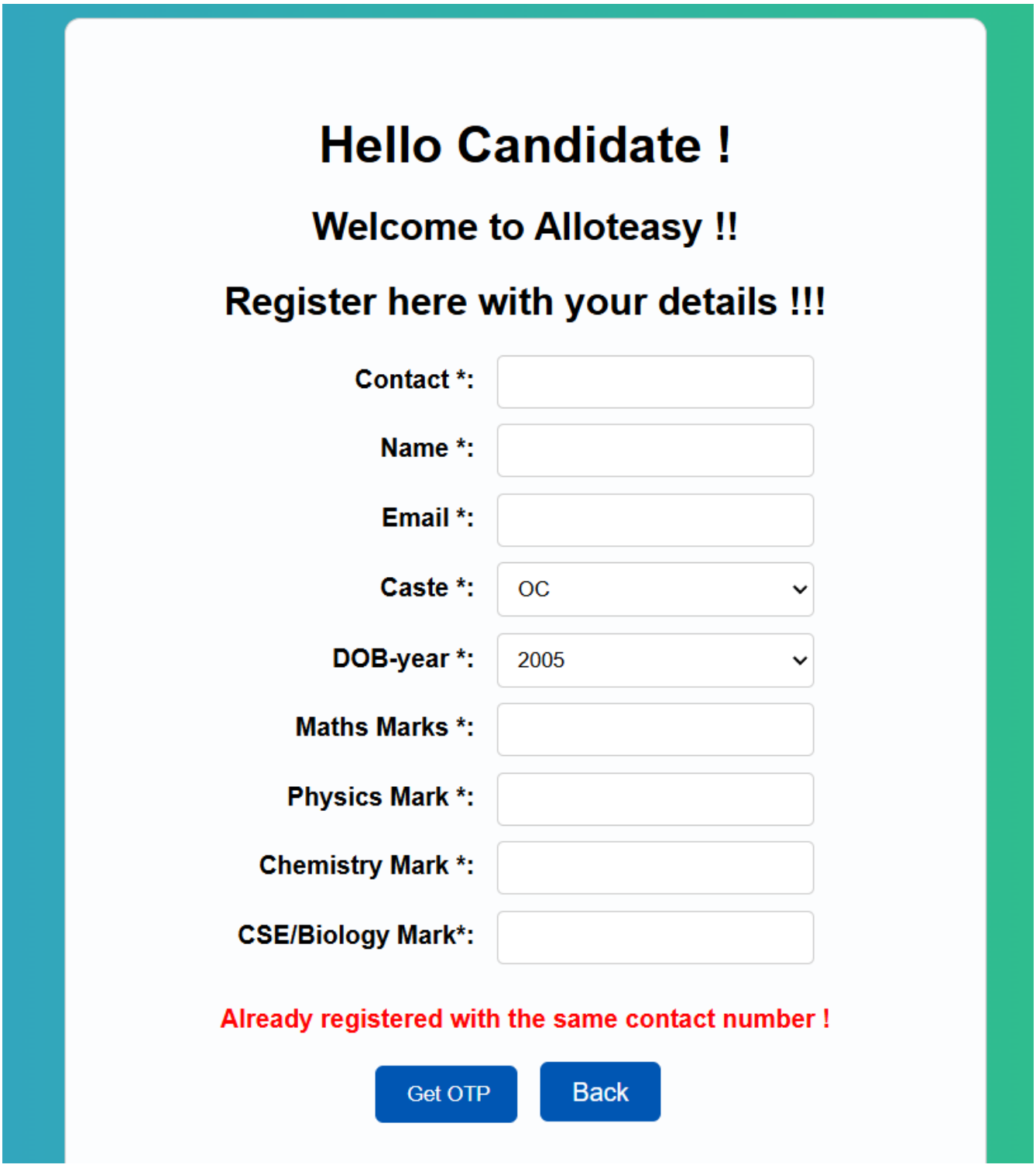
- **Input:** Some random Candidate name, Contact number, Password
- **Expected Output**: " Username doesn't exist ! "
- **Actual Output**:



Registration Module Test Cases:

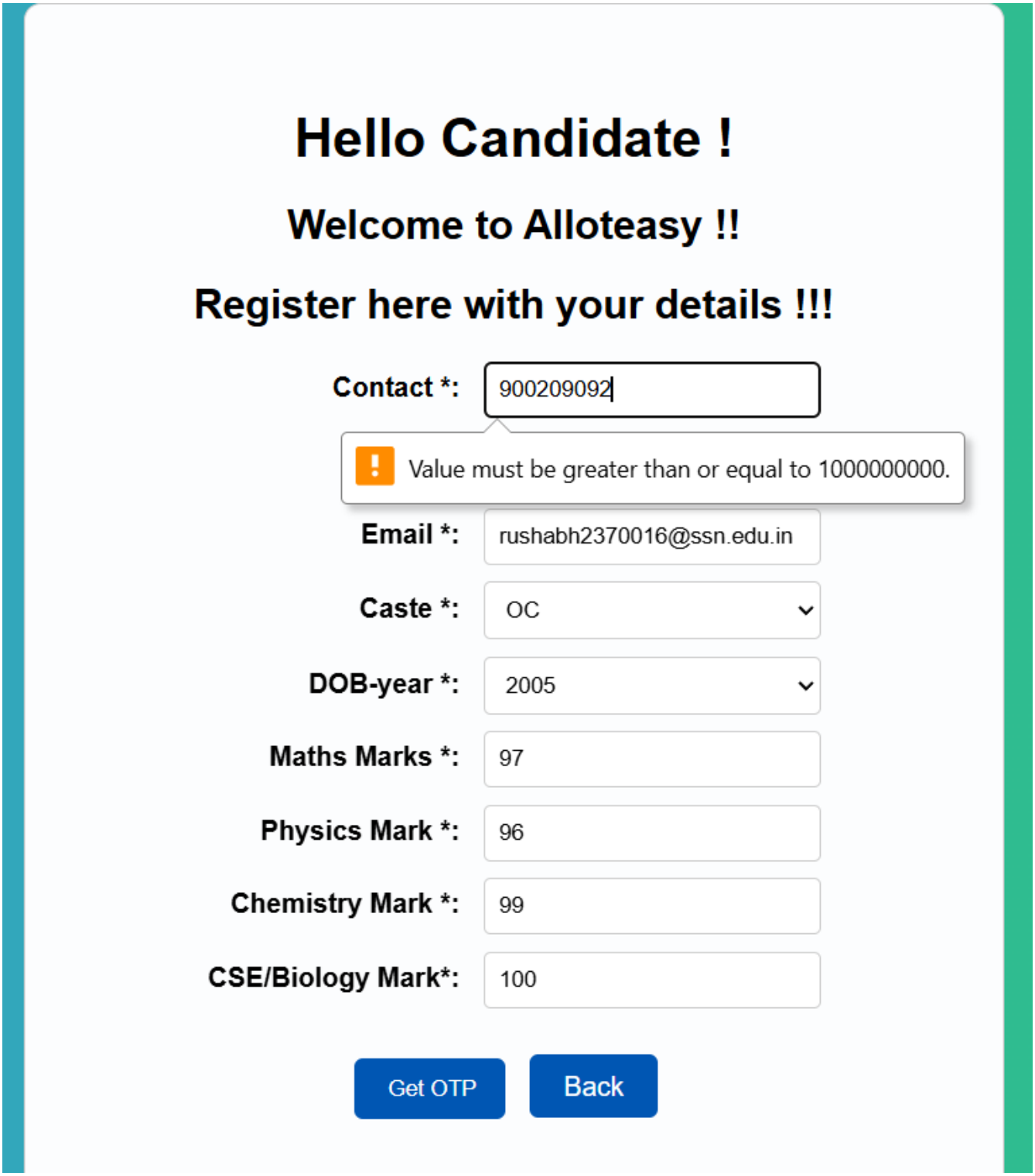
Test Case : Registering with the same contact number already present in the Database

- **Input:** Contact number : "9002090952" which is already present in the user db
- **Expected Output**: " User already registered with the same contact number ! "
- **Actual Output**:


Hello Candidate !
Welcome to Alloteasy !!
Register here with your details !!!
Contact *:
Name *:
Email *:
Caste *:
OC
DOB-year *:
2005
Maths Marks *:
Physics Mark *:
Chemistry Mark *:
CSE/Biology Mark*:
Already registered with the same contact number !
Get OTP
Back

Test Case : Wrong Contact number (digits not equal to 10)

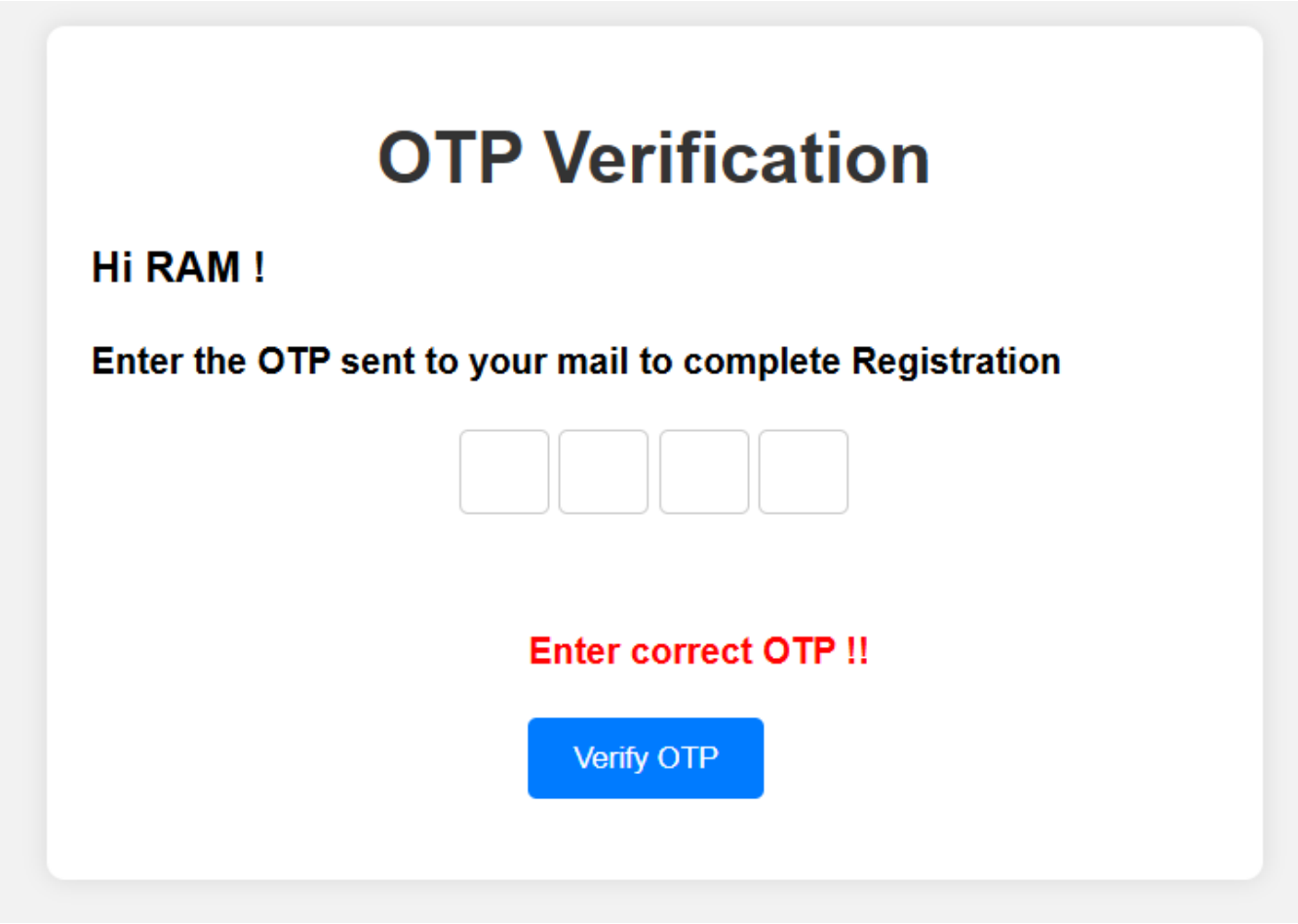
- **Input:** Contact number : “900209092” which is only 9 digit
- **Expected Output**: "Value must be greater than or equal to 1,000,000,000 (10 digits) "
- **Actual Output**:



OTP verification Module Test Cases:

Test Case : Entering wrong otp instead of the otp sent to the mail

- **Input:** OTP : “ 1234 ”
- **Expected Output**: "Enter correct OTP "
- **Actual Output**:



Ranking Module Test Cases:

Test Case : If two user has same details, the priority goes from maths , physics, chemistry marks, DOB year, name and finally,the user registered first gets preference

- **Input:** Same details of one already registered Candidate
- **Expected Output**: " The one who registered first gets ranked first "
- **Actual Output**:

Though every details are same, ‘Chetan Shah’ who registered first gets more preference than ‘Pandi’ who registered then.

User Name	Password	User Email	Contact Number	Caste	Year of birth	Maths mark	Physics mark	Chemistry mark	BIO/CSE mark	Total	Cutoff	New Value	Rank	Delete	View
Chetan Shah	chet2004	chetan_shah45@gmail.com	9071330245	MBC	2004	97	94	97	91	379	192.50	184.80	1	Delete	View
Pandi	pand2004	pandiarajan2370062@ssn.edu.in	1234567890	MBC	2004	97	94	97	91	379	192.50	184.80	2	Delete	View

Allottment Module Test Cases:

Test Case : First rank Candidate should be allotted with his First Preference if that seat is vacant.

- **Input:** Some random Choice filling for the first rank candidate.
- **Expected Output**: " First Preference Should be Allotted "
- **Actual Output**:

Name:

Chetan Shah

Contact number:

9071330245

Cutoff:

192.50

Rank:

1

First Preference:

MTECH

Second Preference:

BME

Third Preference:

ECE

Allotted Department :

MTECH

Back

Key Aspects:

- We are using MySql database (two datatables) to protect our data.
- In order to work on large scale data, we are using Random Generation module to populate the database which enables us to work on large database and reflects our software ability to work on multiple datasets.
- Our Software is Scalable and it can be updated with College information and Seat Matrix to Allot Seats for Multiple College.
- Ranking by Sorting done on MySql database using Python connectivity which makes our program execute faster.
- Ranking is based on Merit (both Cutoff and caste) which reflects the Practical Counselling softwares.

Limitation of the solution Provided:

- No data of real time users.
- In choice filling there is only one college available and the total number of choices are only nine.
- The seats are very much limited.
- The project has not been deployed in internet for the access of other users.
- The Contact number (username) cannot be same for two or more members.
- The password is fixed which leads to misuse.

Alternate Solution:

Instead of Rank for Seat Allocation, we can use Percentile method for all Candidates to allocate seats for them. The Percentile is calculated as follows

$$\text{Percentile} = \frac{(\text{Total Candidate} - \text{Rank of the Candidate})}{(\text{Total Candidates})} \times 100$$

Concerns with respect to society, legal and ethical perspectives:

1. **Fairness and Equality:**

The system or the program has a fair way of ranking. It does not have discrimination based on gender, race, etc.

2. **Transparency:**

The process of seat allocation is transparent, with clear criteria which must be known to students, faculty and parents.

3. **Protecting Data Privacy:**

The system or the program gives priority or importance to data privacy ensuring that the student data is confidential or private. This include storing the data properly and securely and ensuring no data leak or access to data without admin id and password.

4. **Responsible Data Usage:**

The system or program collects data and stores that is required only for the allocation and ranking process.

5. **Improved opportunities:**

The people who have been deprived of opportunities for a long period of time and under privileged people have been given more preference.

Learning Outcomes:

Problem solving Skills:

Through the project we could enhance our problem-solving abilities by identifying and addressing challenges related to user input validation, ranking and sorting, seat allocation for different test cases.

Collaboration and Communication:

This fostered collaboration and effective communication among team members as they work together to, divide tasks, integrate individual components, and ensure smooth functioning of the Engineering seat allocation system.

Project management:

Enables us to develop project management skills such as setting timelines, allocating resources effectively, and adapting to changing requirements, which contributes to a better understanding of the software development and project delivery.

Technical proficiency:

We could develop a solid understanding of programming concepts and techniques by implementing various functionalities of Engineering seat allocation system, such as ranking, seat allocation and user input validation.

Summary:

The software starts with a login system with username and password verification. Users can create new accounts with personal details. Also, there is a random generation of candidate information to facilitate the software to work on large amount of data. Upon login or random generation, total marks, cutoff, and a new value are calculated and stored. Admin can generate a rank list based on candidate details (new value), by sorting. Users can view their individual results, while admins can access a comprehensive rank list of all candidates.

References:

https://ieeexplore.ieee.org/document/6208616

https://www.researchgate.net/publication/261212549_An_optimized_e-Allotment_algorithm_for_admissions_in_educational_institutes

https://vitalflux.com/ranking-algorithms-types-concepts-examples/